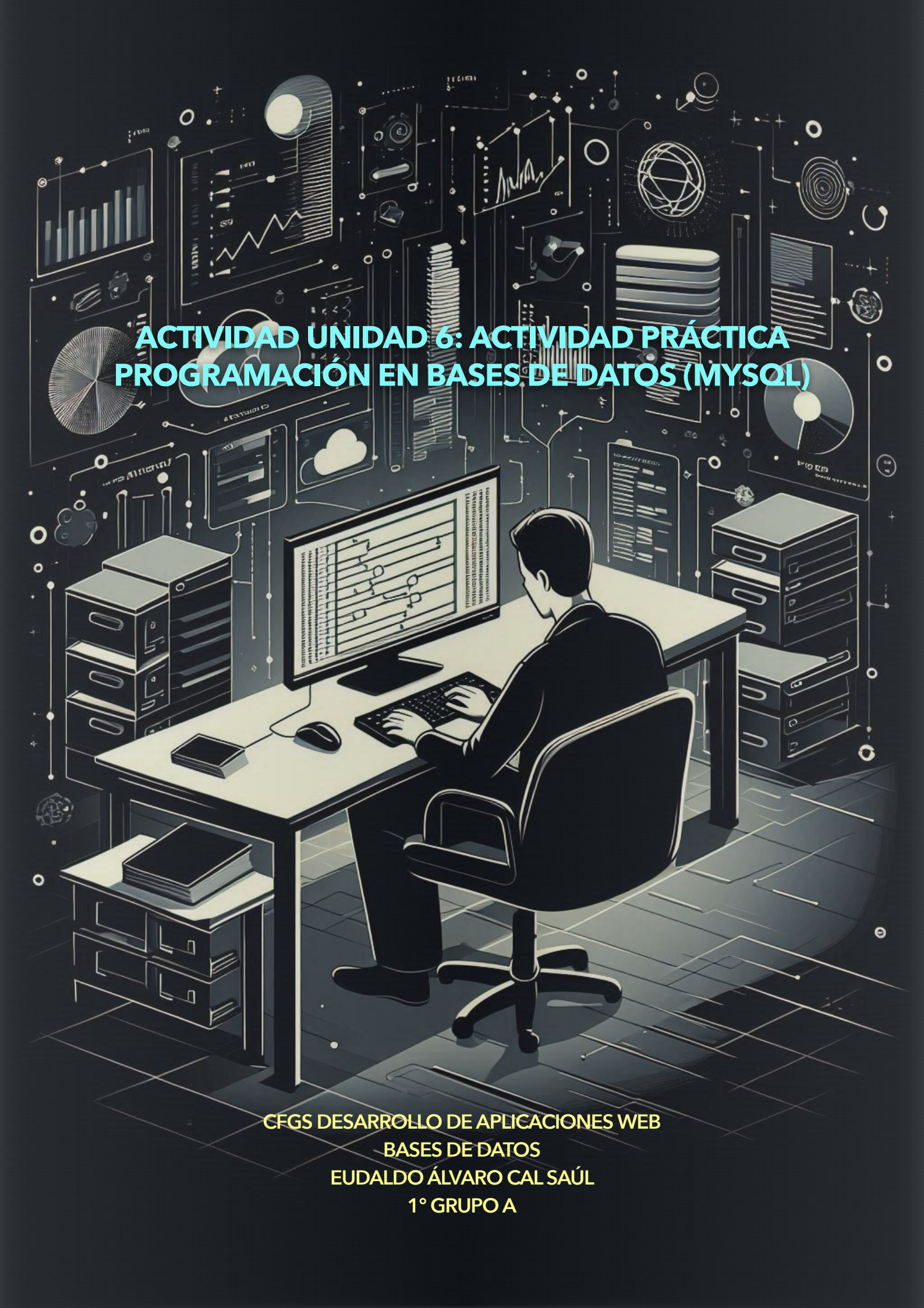




ACTIVIDAD UNIDAD 6: ACTIVIDAD PRÁCTICA PROGRAMACIÓN EN BASES DE DATOS (MYSQL)

CFGS DESARROLLO DE APLICACIONES WEB
BASES DE DATOS
EUDALDO ÁLVARO CAL SAÚL
1º GRUPO A

Indice

Actividad 1. Procedimiento de reasignación de agentes	2
Actividad 2. Disparador de validación	6
Casos de prueba sobre el trigger validar_agentes:	8
Casos de prueba sobre el trigger validar_agentes_update:	10
Pregunta adicional	10

ACTIVIDAD 1. PROCEDIMIENTO DE REASIGNACIÓN DE AGENTES

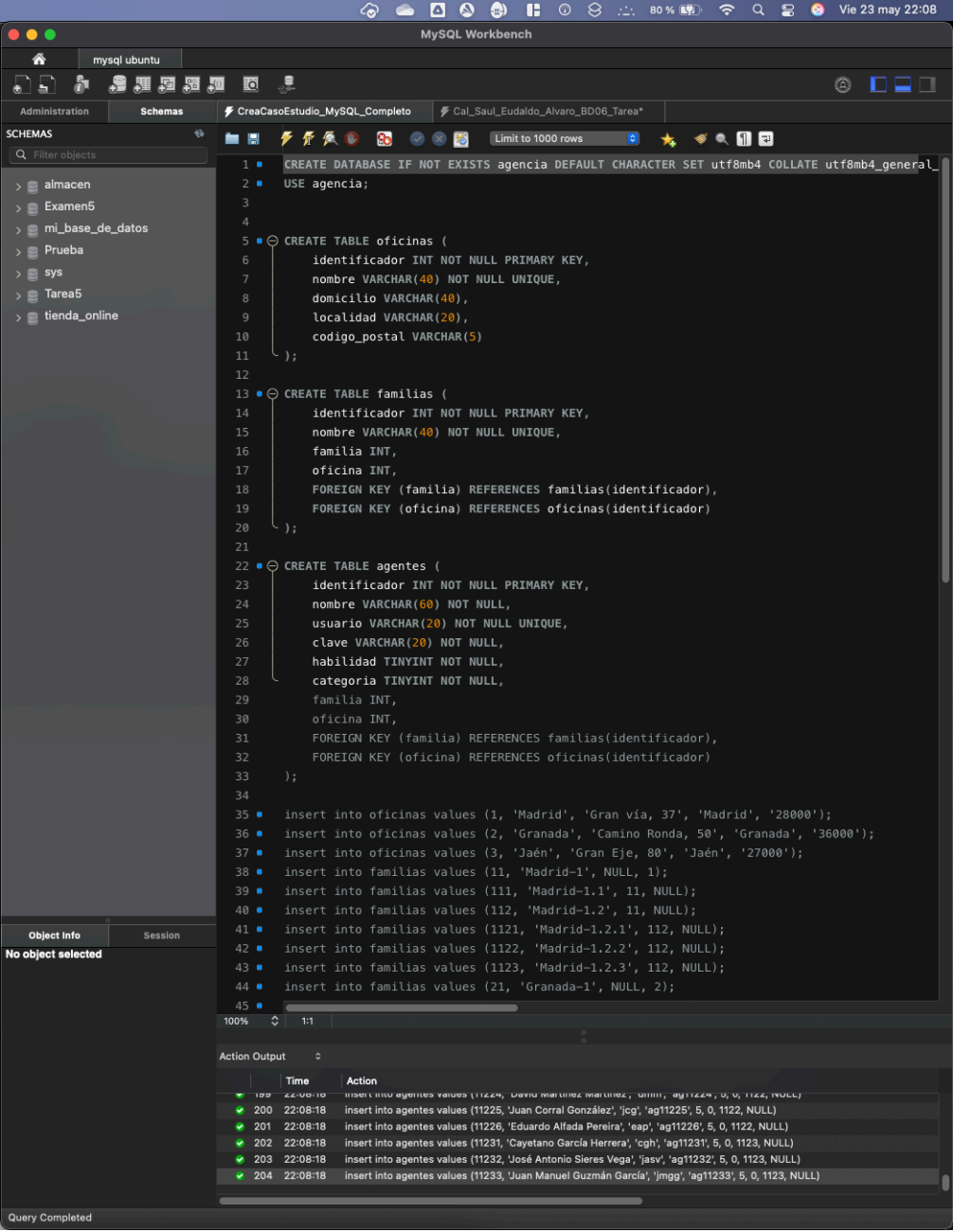
Crea un procedimiento llamado `CambiarAgentesFamilia` que realice lo siguiente:

- Verifica que **ambas familias existen** (tabla `familias`).
- Comprueba que **no son iguales**.
- Actualiza la familia de todos los agentes que pertenecen a la familia origen.
- Muestra un mensaje del tipo:
"Se han trasladado XXX agentes de la familia XXXXXX a la familia ZZZZZZ"

Debes usar:

- `SIGNAL SQLSTATE '45000'` para errores personalizados
- Funciones como `SELECT INTO`, `ROW_COUNT()`, `CONCAT()`

Como paso previo cargamos el script para crear la base de datos, tablas y la inserción de datos en las tablas.



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view containing: almacen, Examen5, mi_base_de_datos, Prueba, sys, Tarea5, and tienda_online. The main editor area shows a SQL script with the following content:

```
1 CREATE DATABASE IF NOT EXISTS agencia DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
2 USE agencia;
3
4
5 CREATE TABLE oficinas (
6     identificador INT NOT NULL PRIMARY KEY,
7     nombre VARCHAR(40) NOT NULL UNIQUE,
8     domicilio VARCHAR(40),
9     localidad VARCHAR(20),
10    codigo_postal VARCHAR(5)
11 );
12
13 CREATE TABLE familias (
14     identificador INT NOT NULL PRIMARY KEY,
15     nombre VARCHAR(40) NOT NULL UNIQUE,
16     familia INT,
17     oficina INT,
18     FOREIGN KEY (familia) REFERENCES familias(identificador),
19     FOREIGN KEY (oficina) REFERENCES oficinas(identificador)
20 );
21
22 CREATE TABLE agentes (
23     identificador INT NOT NULL PRIMARY KEY,
24     nombre VARCHAR(60) NOT NULL,
25     usuario VARCHAR(20) NOT NULL UNIQUE,
26     clave VARCHAR(20) NOT NULL,
27     habilidad TINYINT NOT NULL,
28     categoria TINYINT NOT NULL,
29     familia INT,
30     oficina INT,
31     FOREIGN KEY (familia) REFERENCES familias(identificador),
32     FOREIGN KEY (oficina) REFERENCES oficinas(identificador)
33 );
34
35 insert into oficinas values (1, 'Madrid', 'Gran vía, 37', 'Madrid', '28000');
36 insert into oficinas values (2, 'Granada', 'Camino Ronda, 50', 'Granada', '36000');
37 insert into oficinas values (3, 'Jaén', 'Gran Eje, 80', 'Jaén', '27000');
38 insert into familias values (11, 'Madrid-1', NULL, 1);
39 insert into familias values (111, 'Madrid-1.1', 11, NULL);
40 insert into familias values (112, 'Madrid-1.2', 11, NULL);
41 insert into familias values (1121, 'Madrid-1.2.1', 112, NULL);
42 insert into familias values (1122, 'Madrid-1.2.2', 112, NULL);
43 insert into familias values (1123, 'Madrid-1.2.3', 112, NULL);
44 insert into familias values (21, 'Granada-1', NULL, 2);
45
```

The bottom panel shows the 'Action Output' table with the following data:

	Time	Action
✓	22:08:18	insert into agentes values (11224, 'David Martínez Martínez', 'dmr', 'ag11224', 5, 0, 1122, NULL)
✓	200 22:08:18	insert into agentes values (11225, 'Juan Corral González', 'jcg', 'ag11225', 5, 0, 1122, NULL)
✓	201 22:08:18	insert into agentes values (11226, 'Eduardo Alfada Pereira', 'eap', 'ag11226', 5, 0, 1122, NULL)
✓	202 22:08:18	insert into agentes values (11231, 'Cayetano García Herrera', 'cgh', 'ag11231', 5, 0, 1123, NULL)
✓	203 22:08:18	insert into agentes values (11232, 'José Antonio Sieres Vega', 'jasv', 'ag11232', 5, 0, 1123, NULL)
✓	204 22:08:18	insert into agentes values (11233, 'Juan Manuel Guzmán García', 'jmgg', 'ag11233', 5, 0, 1123, NULL)

The status bar at the bottom indicates 'Query Completed'.

Ejecutamos el script del procedimiento para crear el procedimiento en la base de datos.

The screenshot shows the MySQL Workbench interface. On the left, the 'Schemas' pane shows a database named 'agencia' with several tables and views. The main editor window displays the SQL script for creating the stored procedure 'CambiarAgentesFamilia'. The script includes comments in Spanish describing the procedure's purpose: to change the family of agents. It defines variables for origin and destination family IDs, existence checks, and names. It includes validation logic to ensure the origin and destination families exist and are not the same. Finally, it includes a query to retrieve the names of the families involved.

```

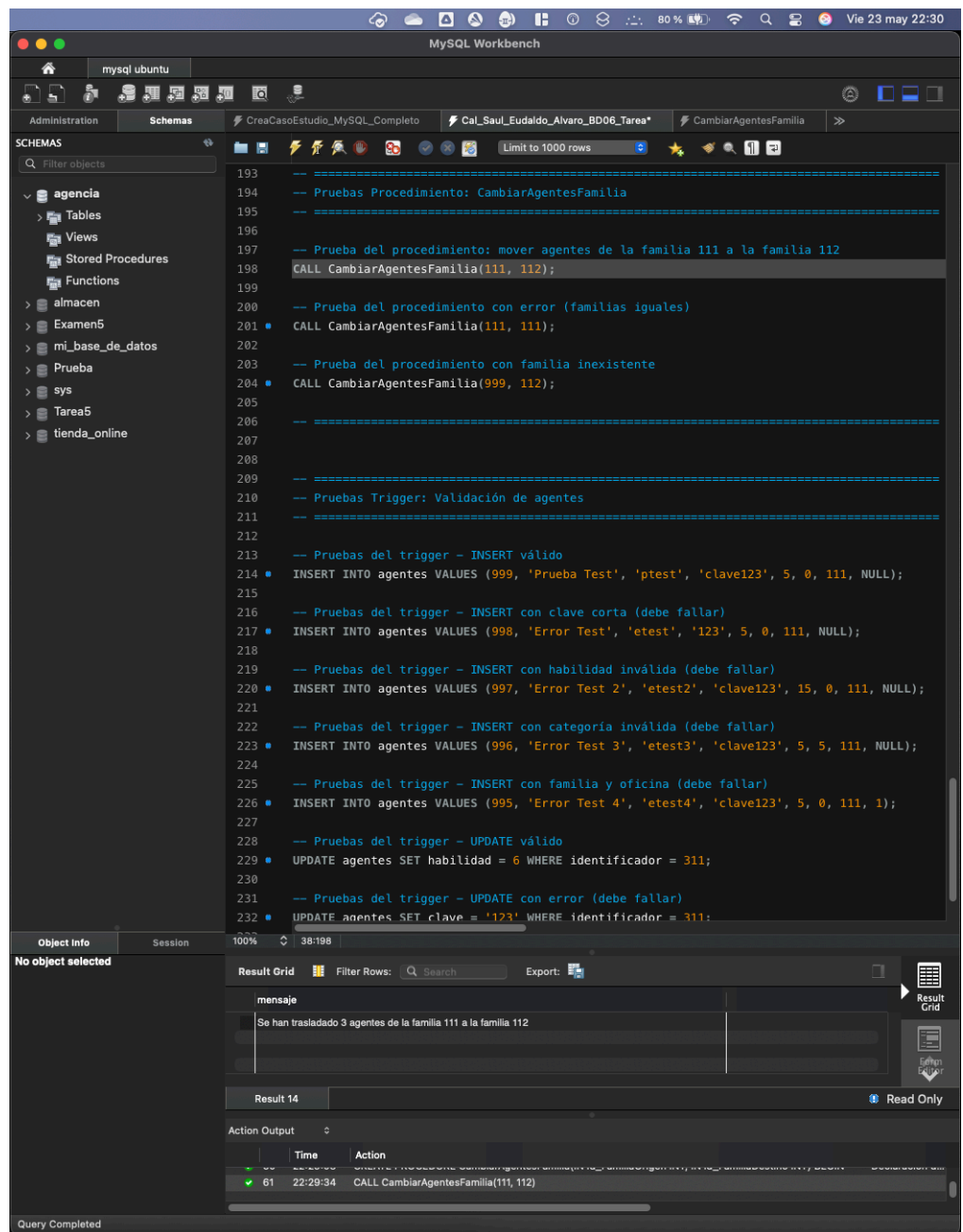
7 DELIMITER //
8
9 -- =====
10 -- Procedimiento: CambiarAgentesFamilia
11 -- Descripción: Cambia los agentes de una familia a otra
12 -- =====
13 CREATE PROCEDURE CambiarAgentesFamilia(IN id_FamiliaOrigen INT, IN id_FamiliaDestino INT)
14 BEGIN
15     -- Declaración de variables
16     DECLARE v_existe_origen INT DEFAULT 0;
17     DECLARE v_existe_destino INT DEFAULT 0;
18     DECLARE v_nombre_origen VARCHAR(40);
19     DECLARE v_nombre_destino VARCHAR(40);
20     DECLARE v_agentes_afectados INT DEFAULT 0;
21
22     -- Validación de igualdad entre ids
23     IF id_FamiliaOrigen = id_FamiliaDestino THEN
24         SIGNAL SQLSTATE '45000'
25         SET MESSAGE_TEXT = 'Error: La familia origen y destino no pueden ser iguales';
26     END IF;
27
28     -- Validación de existencia de familias
29     SELECT COUNT(*) INTO v_existe_origen
30     FROM familias
31     WHERE identificador = id_FamiliaOrigen;
32
33     SELECT COUNT(*) INTO v_existe_destino
34     FROM familias
35     WHERE identificador = id_FamiliaDestino;
36
37     IF v_existe_origen = 0 THEN
38         SIGNAL SQLSTATE '45000'
39         SET MESSAGE_TEXT = 'Error: La familia origen no existe';
40     END IF;
41
42     IF v_existe_destino = 0 THEN
43         SIGNAL SQLSTATE '45000'
44         SET MESSAGE_TEXT = 'Error: La familia destino no existe';
45     END IF;
46
47     -- Obtener nombres de las familias (origen y destino)
48     SELECT nombre INTO v_nombre_origen
49     FROM familias
50     WHERE identificador = id_FamiliaOrigen;
51
52 
```

The 'Action Output' pane at the bottom shows the execution results of the script. It lists five actions, all of which were successful (indicated by green checkmarks). The actions include inserting data into the 'agentes' table and creating the 'CambiarAgentesFamilia' stored procedure.

	Time	Action
✓ 200	22:08:18	insert into agentes values (11226, 'Juan Carlos Gonzalez', 'jcg', 'ag11226', 5, 0, 1122, NULL)
✓ 201	22:08:18	insert into agentes values (11226, 'Eduardo Alfada Pereira', 'eap', 'ag11226', 5, 0, 1122, NULL)
✓ 202	22:08:18	insert into agentes values (11231, 'Cayetano García Herrera', 'cgh', 'ag11231', 5, 0, 1123, NULL)
✓ 203	22:08:18	insert into agentes values (11232, 'José Antonio Sieres Vega', 'jasv', 'ag11232', 5, 0, 1123, NULL)
✓ 204	22:08:18	insert into agentes values (11233, 'Juan Manuel Guzmán García', 'jmgg', 'ag11233', 5, 0, 1123, NULL)
✓ 205	22:12:07	CREATE PROCEDURE CambiarAgentesFamilia(IN id_FamiliaOrigen INT, IN id_FamiliaDestino INT) BEGIN -- Declaración d...

Query Completed

Ejecutamos el primer caso de prueba y nos devuelve el resultado esperado.



Ejecutamos el segundo caso de prueba y nos devuelve el resultado esperado.

Action Output

	Time	Action	Response	Duration / Fetc
✓ 60	22:29:03	CREATE PROCEDURE CambiarAgentesFamilia(IN id_FamiliaOrig...	0 row(s) affected	0.0052 sec
✓ 61	22:29:34	CALL CambiarAgentesFamilia(111, 112)	1 row(s) returned	0.0028 sec / 0.
✗ 62	22:30:08	CALL CambiarAgentesFamilia(111, 111)	Error Code: 1644. Error: La familia origen y destino no pueden ser iguales	0.00045 sec

Query interrupted

Ejecutamos el tercer caso de prueba y nos devuelve el resultado esperado.

Action Output

	Time	Action	Response	Duration / Fetc	
✓	60	22:29:03	CREATE PROCEDURE CambiarAgentesFamilia(IN id_FamiliaOrig...	0 row(s) affected	0.0052 sec
✓	61	22:29:34	CALL CambiarAgentesFamilia(111, 112)	1 row(s) returned	0.0028 sec / 0.
✗	62	22:30:08	CALL CambiarAgentesFamilia(111, 111)	Error Code: 1644. Error: La familia origen y destino no pueden ser iguales	0.00045 sec

Query interrupted

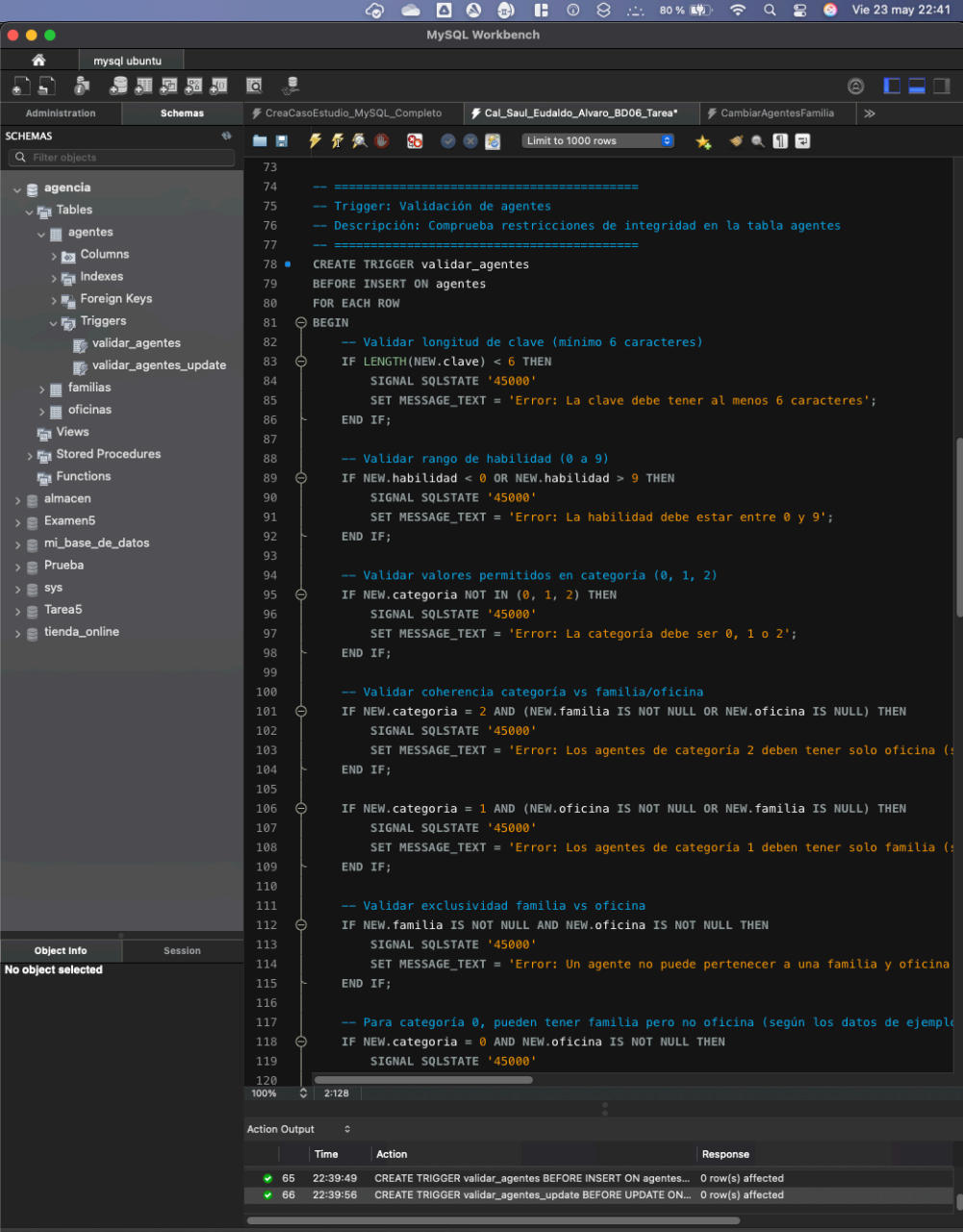
ACTIVIDAD 2. DISPARADOR DE VALIDACIÓN

Crea un **trigger BEFORE INSERT OR UPDATE** sobre la tabla **agentes** que verifique:

- Longitud mínima de **clave**: 6 caracteres
- **habilidad**: entre 0 y 9
- **categoria**: sólo 0, 1 o 2
- Si **categoria** = 2: debe tener **solo oficina**
- Si **categoria** = 1: debe tener **solo familia**
- Un agente no puede tener familia y oficina a la vez

Debes lanzar errores con mensajes significativos usando **SIGNAL SQLSTATE**.

Ejecutamos la creación del trigger `validar_agentes` y observamos que se ejecuta correctamente.



```
73
74 -- =====
75 -- Trigger: Validación de agentes
76 -- Descripción: Comprueba restricciones de integridad en la tabla agentes
77 -- =====
78 CREATE TRIGGER validar_agentes
79 BEFORE INSERT ON agentes
80 FOR EACH ROW
81 BEGIN
82     -- Validar longitud de clave (mínimo 6 caracteres)
83     IF LENGTH(NEW.clave) < 6 THEN
84         SIGNAL SQLSTATE '45000'
85         SET MESSAGE_TEXT = 'Error: La clave debe tener al menos 6 caracteres';
86     END IF;
87
88     -- Validar rango de habilidad (0 a 9)
89     IF NEW.habilidad < 0 OR NEW.habilidad > 9 THEN
90         SIGNAL SQLSTATE '45000'
91         SET MESSAGE_TEXT = 'Error: La habilidad debe estar entre 0 y 9';
92     END IF;
93
94     -- Validar valores permitidos en categoria (0, 1, 2)
95     IF NEW.categoria NOT IN (0, 1, 2) THEN
96         SIGNAL SQLSTATE '45000'
97         SET MESSAGE_TEXT = 'Error: La categoria debe ser 0, 1 o 2';
98     END IF;
99
100     -- Validar coherencia categoria vs familia/oficina
101     IF NEW.categoria = 2 AND (NEW.familia IS NOT NULL OR NEW.oficina IS NULL) THEN
102         SIGNAL SQLSTATE '45000'
103         SET MESSAGE_TEXT = 'Error: Los agentes de categoría 2 deben tener solo oficina (';
104     END IF;
105
106     IF NEW.categoria = 1 AND (NEW.oficina IS NOT NULL OR NEW.familia IS NULL) THEN
107         SIGNAL SQLSTATE '45000'
108         SET MESSAGE_TEXT = 'Error: Los agentes de categoría 1 deben tener solo familia (';
109     END IF;
110
111     -- Validar exclusividad familia vs oficina
112     IF NEW.familia IS NOT NULL AND NEW.oficina IS NOT NULL THEN
113         SIGNAL SQLSTATE '45000'
114         SET MESSAGE_TEXT = 'Error: Un agente no puede pertenecer a una familia y oficina';
115     END IF;
116
117     -- Para categoria 0, pueden tener familia pero no oficina (según los datos de ejemplo)
118     IF NEW.categoria = 0 AND NEW.oficina IS NOT NULL THEN
119         SIGNAL SQLSTATE '45000'
120
121
122
```

Time	Action	Response
65 22:39:49	CREATE TRIGGER validar_agentes BEFORE INSERT ON agentes...	0 row(s) affected
66 22:39:56	CREATE TRIGGER validar_agentes_update BEFORE UPDATE ON...	0 row(s) affected

Query Completed

Ejecutamos la creación del trigger `validar_agentes_update` y observamos que se ejecuta correctamente.

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows a tree view with 'agencia' expanded, containing 'Tables', 'Columns', 'Indexes', 'Foreign Keys', 'Triggers', 'Views', 'Stored Procedures', and 'Functions'. The 'Triggers' folder is selected, showing 'validar_agentes' and 'validar_agentes_update'. The main editor shows the SQL code for creating the trigger. The 'Action Output' pane at the bottom shows the execution results.

```
127 -- Trigger adicional para UPDATE
128 CREATE TRIGGER validar_agentes_update
129 BEFORE UPDATE ON agentes
130 FOR EACH ROW
131 BEGIN
132     -- Validar longitud de clave solo si se está modificando
133     IF NEW.clave != OLD.clave AND LENGTH(NEW.clave) < 6 THEN
134         SIGNAL SQLSTATE '45000'
135         SET MESSAGE_TEXT = 'Error: La clave debe tener al menos 6 caracteres';
136     END IF;
137
138     -- Validar rango de habilidad (0 a 9)
139     IF NEW.habilidad < 0 OR NEW.habilidad > 9 THEN
140         SIGNAL SQLSTATE '45000'
141         SET MESSAGE_TEXT = 'Error: La habilidad debe estar entre 0 y 9';
142     END IF;
143
144     -- Validar valores permitidos en categoría (0, 1, 2)
145     IF NEW.categoria NOT IN (0, 1, 2) THEN
146         SIGNAL SQLSTATE '45000'
147         SET MESSAGE_TEXT = 'Error: La categoría debe ser 0, 1 o 2';
148     END IF;
149
150     -- Validar coherencia categoría vs familia/oficina
151     IF NEW.categoria = 2 AND (NEW.familia IS NOT NULL OR NEW.oficina IS NULL) THEN
152         SIGNAL SQLSTATE '45000'
153         SET MESSAGE_TEXT = 'Error: Los agentes de categoría 2 deben tener solo oficina (o familia)';
154     END IF;
155
156     IF NEW.categoria = 1 AND (NEW.oficina IS NOT NULL OR NEW.familia IS NULL) THEN
157         SIGNAL SQLSTATE '45000'
158         SET MESSAGE_TEXT = 'Error: Los agentes de categoría 1 deben tener solo familia (o oficina)';
159     END IF;
160
161     -- Validar exclusividad familia vs oficina
162     IF NEW.familia IS NOT NULL AND NEW.oficina IS NOT NULL THEN
163         SIGNAL SQLSTATE '45000'
164         SET MESSAGE_TEXT = 'Error: Un agente no puede pertenecer a una familia y oficina';
165     END IF;
166
167     -- Para categoría 0, pueden tener familia pero no oficina
168     IF NEW.categoria = 0 AND NEW.oficina IS NOT NULL THEN
169         SIGNAL SQLSTATE '45000'
170         SET MESSAGE_TEXT = 'Error: Los agentes de categoría 0 no pueden tener oficina asignada';
171     END IF;
172
173 END;
174
```

	Time	Action	Response
✓	126 23:17:22	CREATE TRIGGER validar_agentes BEFORE INSERT ON agentes FOR EACH ROW...	0 row(s) affected
✓	127 23:17:25	CREATE TRIGGER validar_agentes_update BEFORE UPDATE ON agentes FOR E...	0 row(s) affected

Query Completed

CASOS DE PRUEBA SOBRE EL TRIGGER VALIDAR_AGENTES:

Ejecutamos el primer caso de prueba y nos devuelve el resultado esperado.

```
190
191 */
192
193 -- =====
194 -- Pruebas Procedimiento: CambiarAgentesFamilia
195 -- =====
196
197 -- Prueba del procedimiento: mover agentes de la familia 111 a la familia 112
198 CALL CambiarAgentesFamilia(111, 112);
199
200 -- Prueba del procedimiento con error (familias iguales)
201 CALL CambiarAgentesFamilia(111, 111);
202
203 -- Prueba del procedimiento con familia inexistente
204 CALL CambiarAgentesFamilia(999, 112);
205
206 -- =====
207
208 -- =====
209 -- Pruebas Trigger: Validación de agentes
210 -- =====
211
212
213 -- Pruebas del trigger - INSERT válido
214 INSERT INTO agentes VALUES (999, 'Prueba Test', 'ptest', 'clave123', 5, 0, 111, NULL);
215
216 -- Pruebas del trigger - INSERT con clave corta (debe fallar)
217 INSERT INTO agentes VALUES (998, 'Error Test', 'etest', '123', 5, 0, 111, NULL);
218
219 -- Pruebas del trigger - INSERT con habilidad inválida (debe fallar)
220 INSERT INTO agentes VALUES (997, 'Error Test 2', 'etest2', 'clave123', 15, 0, 111, NULL);
221
222 -- Pruebas del trigger - INSERT con categoría inválida (debe fallar)
223 INSERT INTO agentes VALUES (996, 'Error Test 3', 'etest3', 'clave123', 5, 5, 111, NULL);
224
225 -- Pruebas del trigger - INSERT con familia y oficina (debe fallar)
226 INSERT INTO agentes VALUES (995, 'Error Test 4', 'etest4', 'clave123', 5, 0, 111, 1);
227
228 -- Pruebas del trigger - UPDATE válido
229 UPDATE agentes SET habilidad = 6 WHERE identificador = 311;
230
231 -- Pruebas del trigger - UPDATE con error (debe fallar)
232 UPDATE agentes SET clave = '123' WHERE identificador = 311;
233
234 -- =====
235
236
```

	Time	Action	Response	Duration / Fe
65	22:39:49	CREATE TRIGGER validar_agentes BEFORE INSERT ON agentes FOR EACH ROW BEGIN -- Validar longitud de clave (mínimo 6 car...	0 row(s) affected	0.0055 sec
66	22:39:56	CREATE TRIGGER validar_agentes_update BEFORE UPDATE ON agentes FOR EACH ROW BEGIN -- Validar longitud de clave (mín...	0 row(s) affected	0.0043 sec
67	22:46:27	INSERT INTO agentes VALUES (999, 'Prueba Test', 'ptest', 'clave123', 5, 0, 111, NULL)	1 row(s) affected	0.0050 sec

Query Completed

Ejecutamos el segundo caso de prueba y nos devuelve el resultado esperado.

```
216 -- Pruebas del trigger - INSERT con clave corta (debe fallar)
217 INSERT INTO agentes VALUES (998, 'Error Test', 'etest', '123', 5, 0, 111, NULL);
```

	Time	Action	Response	Duration / Fe
66	22:39:56	CREATE TRIGGER validar_agentes_update BEFORE UPDATE ON agentes FOR E...	0 row(s) affected	0.0043 sec
67	22:46:27	INSERT INTO agentes VALUES (999, 'Prueba Test', 'ptest', 'clave123', 5, 0, 111,...	1 row(s) affected	0.0050 sec
68	22:48:02	INSERT INTO agentes VALUES (998, 'Error Test', 'etest', '123', 5, 0, 111, NULL)	Error Code: 1644. Error: La clave debe tener al menos 6 caracteres	0.00056 sec

Query interrupted

Ejecutamos el tercer caso de prueba y nos devuelve el resultado esperado.

```
219 -- Pruebas del trigger - INSERT con habilidad inválida (debe fallar)
220 INSERT INTO agentes VALUES (997, 'Error Test 2', 'etest2', 'clave123', 15, 0, 111, NULL);
```

	Time	Action	Response	Duration / Fe
✖ 68	22:48:02	INSERT INTO agentes VALUES (998, 'Error Test', 'etest', '123', 5, 0, 111, NULL)	Error Code: 1644. Error: La clave debe tener al menos 6 caracteres	0.00056 sec
✖ 69	22:51:22	INSERT INTO agentes VALUES (997, 'Error Test 2', 'etest2', 'clave123', 15, 0, 111, ...	Error Code: 1644. Error: La habilidad debe estar entre 0 y 9	0.00049 sec

Query interrupted

Ejecutamos el cuarto caso de prueba y nos devuelve el resultado esperado.

```
222 -- Pruebas del trigger - INSERT con categoría inválida (debe fallar)
223 INSERT INTO agentes VALUES (996, 'Error Test 3', 'etest3', 'clave123', 5, 5, 111, NULL);
```

	Time	Action	Response	Duration / Fe
✖ 69	22:51:22	INSERT INTO agentes VALUES (997, 'Error Test 2', 'etest2', 'clave123', 15, 0, 111, ...	Error Code: 1644. Error: La habilidad debe estar entre 0 y 9	0.00049 sec
✖ 70	22:52:50	INSERT INTO agentes VALUES (996, 'Error Test 3', 'etest3', 'clave123', 5, 5, 111, ...	Error Code: 1644. Error: La categoría debe ser 0, 1 o 2	0.00074 sec

Query interrupted

Ejecutamos el quinto caso de prueba y nos devuelve el resultado esperado.

```
225 -- Pruebas del trigger - INSERT con familia y oficina (debe fallar)
226 INSERT INTO agentes VALUES (995, 'Error Test 4', 'etest4', 'clave123', 5, 0, 111, 1);
```

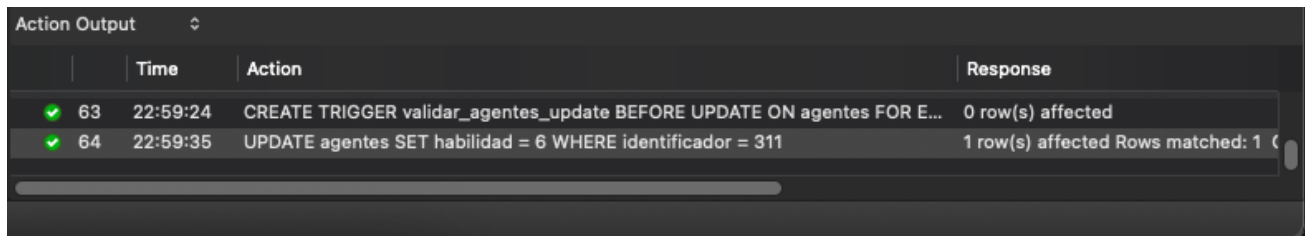
Time	Action	Response
22:52:50	INSERT INTO agentes VALUES (996, 'Error Test 3', 'etest3', 'clave123', 5, 5, 111, ...	Error Code: 1644. Error: La categoría debe ser 0, 1 o 2
22:53:53	INSERT INTO agentes VALUES (995, 'Error Test 4', 'etest4', 'clave123', 5, 0, 111, ...	Error Code: 1644. Error: Un agente no puede pertenecer a una familia y oficina simultáneamente

Query interrupted

CASOS DE PRUEBA SOBRE EL TRIGGER VALIDAR_AGENTES_UPDATE:

Ejecutamos el primer caso de prueba sobre el trigger validar_agentes_update y nos devuelve el resultado esperado.

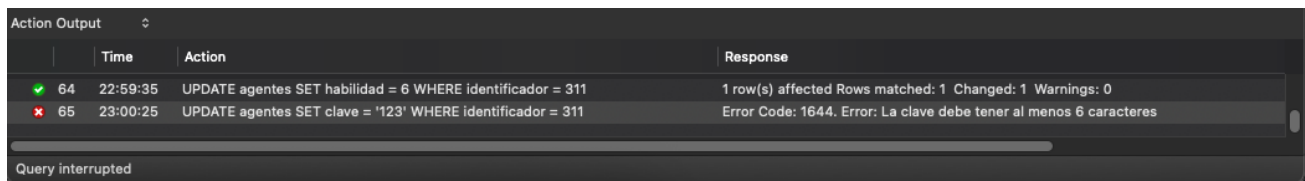
```
228 -- Pruebas del trigger - UPDATE válido
229 UPDATE agentes SET habilidad = 6 WHERE identificador = 311;
```



	Time	Action	Response
✓ 63	22:59:24	CREATE TRIGGER validar_agentes_update BEFORE UPDATE ON agentes FOR E...	0 row(s) affected
✓ 64	22:59:35	UPDATE agentes SET habilidad = 6 WHERE identificador = 311	1 row(s) affected Rows matched: 1

Ejecutamos el segundo caso de prueba y nos devuelve el resultado esperado.

```
231 -- Pruebas del trigger - UPDATE con error (debe fallar)
232 UPDATE agentes SET clave = '123' WHERE identificador = 311;
```



	Time	Action	Response
✓ 64	22:59:35	UPDATE agentes SET habilidad = 6 WHERE identificador = 311	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✗ 65	23:00:25	UPDATE agentes SET clave = '123' WHERE identificador = 311	Error Code: 1644, Error: La clave debe tener al menos 6 caracteres

Query interrupted

PREGUNTA ADICIONAL

Indica qué validaciones podrían definirse con restricciones **CHECK**, **NOT NULL** o **UNIQUE** directamente en el esquema.

Restricciones CHECK

Pueden implementarse directamente en el esquema las validaciones que evalúan:

- Rangos numéricos simples: como verificar que la habilidad esté entre 0 y 9
- Listas de valores permitidos: como validar que la categoría solo tenga valores 0, 1 o 2
- Longitud de cadenas: como verificar que la clave tenga al menos 6 caracteres
- Comparaciones con valores fijos: cualquier validación que compare un campo con constantes

Restricciones NOT NULL

Se aplican a campos que siempre deben tener valor, como nombre, usuario, clave, habilidad y categoría. Son ideales para garantizar la integridad básica de datos obligatorios.

Restricciones UNIQUE

Perfectas para campos que deben ser únicos en toda la tabla, como el usuario de cada agente. También pueden aplicarse a combinaciones de campos.

Limitaciones de las restricciones de esquema

Las validaciones que no pueden implementarse con restricciones CHECK incluyen:

- Lógica condicional compleja que depende de múltiples campos
- Validaciones que requieren consultas a otras tablas
- Reglas de negocio contextual como la exclusividad familia/oficina según categoría
- Validaciones que involucran NULL de manera compleja

Ventajas de usar restricciones de esquema

Las restricciones CHECK, NOT NULL y UNIQUE son más eficientes, se evalúan antes que los triggers, son más claras para documentar el esquema y proporcionan validación automática sin código adicional. Son la primera línea de defensa para la integridad de datos.

Cuándo usar triggers

Los triggers son necesarios únicamente para lógica de negocio que no puede expresarse mediante restricciones simples, especialmente cuando se requiere evaluar combinaciones complejas de campos o implementar reglas contextuales sofisticadas.